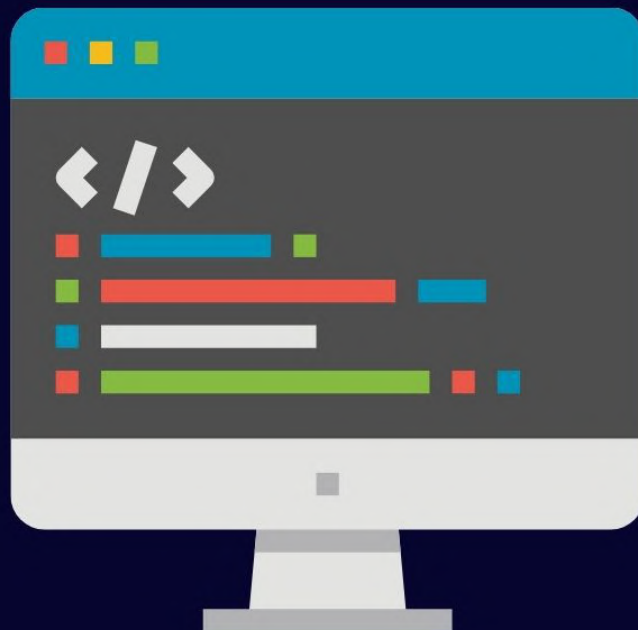


IGCSE COMPUTER SCIENCE 0478 PAPER 2

Scenario Questions Practice



IGCSE PAPER 2 PROGRAMMING QUESTIONS PRACTICE

All the Scenario (12-15 Mark) Questions
Everything available before Feb/March 2023

QUESTION



- 11 A times table test will be set up. Before using the test, the user can set the times table to be tested and the number of questions to be asked. The times tables can be any whole number between 2 and 12 inclusive. The number of questions can be between 5 and 10 inclusive. No questions can be the same in any test. The name of the user should be displayed with all messages and prompts.

Write and test a program that meets the following requirements:

- Sets up the starting conditions for the test
- During the test:
 - displays each question with the question number
 - checks if the answer input is correct and displays the right answer if the answer input was wrong
 - adds one to the user's total every time an answer is right.
- At the end of the test:
 - displays the result
 - asks the user if they want a retest.

You must use pseudocode or program code **and** add comments to explain how your code works. All inputs and outputs must contain suitable messages.

- 13** The 1D array `StudentName[]` contains the names of students in a class. The 2D array `StudentMark[]` contains the mark for each subject, for each student. The position of each student's data in the two arrays is the same, for example, the student in position 10 in `StudentName[]` and `StudentMark[]` is the same.

The variable `ClassSize` contains the number of students in the class. The variable `SubjectNo` contains the number of subjects studied. All students study the same number of subjects.

The arrays and variables have already been set up and the data stored.

Students are awarded a grade based on their average mark.

Average mark	Grade awarded
greater than or equal to 70	distinction
greater than or equal to 55 and less than 70	merit
greater than or equal to 40 and less than 55	pass
less than 40	fail

Write a program that meets the following requirements:

- calculates the combined total mark for each student for all their subjects
- calculates the average mark for each student for all their subjects, rounded to the nearest whole number
- outputs for each student:
 - name
 - combined total mark
 - average mark
 - grade awarded
- calculates, stores and outputs the number of distinctions, merits, passes and fails for the whole class.

You must use pseudocode or program code **and** add comments to explain how your code works.

You do **not** need to initialise the data in the array.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

- [illegible]

12 You can become a member of the town guild if you meet all the following requirements:

- » aged between 18 and 65
- » you have lived in the town for 20 years
or you have lived in the town for 5 years and both your parents still live in the town
or one parent has died and the other parent still lives in the town
or you and a brother or a sister have lived in the town for 10 years
- » you have half a million dollars in your bank account
- » you have never been to prison
- » at least two guild members support your application
- » if five guild members support your application then you could be excused one of the above requirements, apart from age

Write a program that allows you to input answers to questions to ensure that you meet the requirements, issues a unique membership number if you do or explains all the reasons for failure if you don't meet the requirements.

[15]

For IGCSE Computer Science, you should be able to write an understandable algorithm to solve a problem. Your algorithm should be easy to read and understand and may use any of the concepts in this chapter and Chapter 7.

Here are some further problems for you to solve:

10 The one-dimensional array **StaffName[]** contains the names of the staff in a department. Another one-dimensional array **StaffPhone[]** contains the phone numbers for the members of staff. A final one-dimensional array **StaffOffice[]** contains the office location for the members of staff. The position of each member of staff's data in the three arrays is the same, for example, the member of staff in position 5 in **StaffPhone[]** and **StaffOffice[]** is the same.

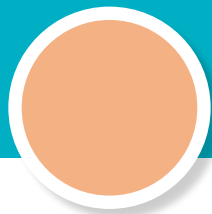
Write a program that meets the following requirements:

- uses procedures to display these lists:
 - staff phone numbers and names in alphabetic order of staff name
 - members of staff grouped by office number
- uses a procedure to display all the details for a member of staff, with the name used as a parameter
- uses a procedure to update the details for a member of staff, with the name used as a parameter.

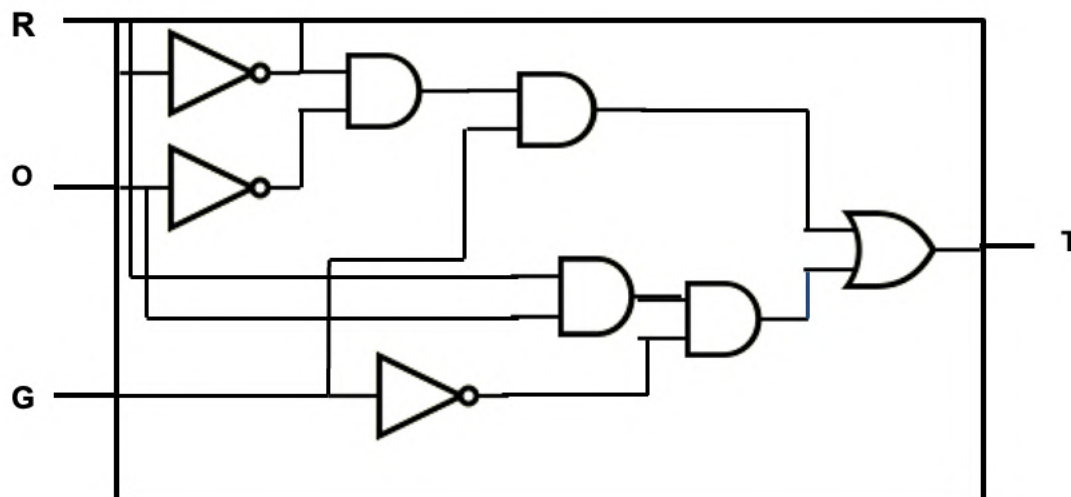
You must use program code with local and global variables and add comments to explain how your code works.

MARKSCHEME





c)



[6]

11

Python:

```

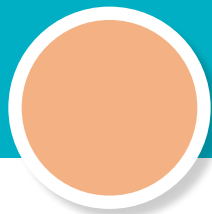
# set up starting conditions
Name = input("Please enter your name ")
Table = 0
while (Table < 1 or Table > 12):
    Table = int(input("Please enter a table between 1 and 12 "))
Questions = 0
while (Questions < 5 or Questions > 10):
    Questions = int(input("Please enter the number of questions between 5 and 10 "))

Retest = True

# conduct the test
while Retest:
    Right = 0
    for QuestionNumber in range(1, Questions + 1):
        print("Question ", QuestionNumber)
        print(Table, " X ", QuestionNumber)
        Answer = int(input("Answer "))
        if Answer == Table*QuestionNumber:
            print(Name, " you are right, well done")
            Right +=1
        else:
            print(Name, " you are wrong the answer is ", Table*QuestionNumber)
    print("Test over ", Name, " you got ", Right, " out of ", Questions)

    Choice = input(" Do you want a retest Y or N? ")# check for a retest
    if Choice != "Y":
        Retest = False
  
```

[14]



Visual Basic:

Module Module1

```
Sub Main()
    Dim Name, Choice As String
    Dim Table, Questions, QuestionNumber, Right, Answer As Integer
    Dim ReTest As Boolean
    ' set up starting conditions
    Console.WriteLine("Please enter your name ")
    Name = Console.ReadLine()
    Do
        Console.WriteLine("Please enter a table between 1 and 12 ")
        Table = Int(Console.ReadLine())
    Loop Until Table >= 1 And Table <= 12

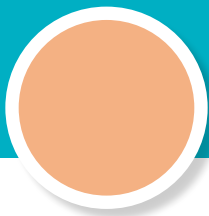
    Do
        Console.WriteLine("Please enter the number between 5 and 10 ")
        Questions = Int(Console.ReadLine())
    Loop Until Questions >= 5 And Questions <= 10

    ReTest = True

    ' conduct the test
    Do
        Right = 0
        For QuestionNumber = 1 To Questions
            Console.WriteLine("Question " & QuestionNumber)
            Console.WriteLine(Table & " X " & QuestionNumber)
            Answer = Int(Console.ReadLine())
            If Answer = QuestionNumber * Table Then
                Console.WriteLine(Name & " you are right well done")
                Right = Right + 1
            Else
                Console.WriteLine(Name & " you are wrong the answer is " & Table *
QuestionNumber)
            End If
        Next
        Console.WriteLine("Test over " & Name & " you got " & Right & " out of " & Questions)
        Console.WriteLine("Do you want a retest Y or N?")
        Choice = Console.ReadLine()
        If Choice <> "Y" Then
            ReTest = False
        End If

        Loop Until Not ReTest
```

[14]



Java:

```
import java.util.Scanner;
class ExamQ11Java
{
    public static void main(String args[])
    {
        // set up starting conditions
        Scanner myObj = new Scanner(System.in);
        String Name;
        System.out.print("Please enter your name ");
        Name = myObj.next();
        int Table;
        do
        {
            System.out.print("Please enter a table between 1 and 12 ");
            Table = myObj.nextInt();
        }
        while (Table < 1 || Table > 12);

        int Questions;
        do
        {
            System.out.print("Please enter the number of questions between 5 and 10 ");
            Questions = myObj.nextInt();
        }
        while (Questions < 5 || Questions > 12);

        boolean Retest = true;

        int QuestionNumber, Answer, Right;

        // conduct the test
        do
        {
            Right = 0;
            for (QuestionNumber= 1; QuestionNumber <= Questions; QuestionNumber++)
            {
                System.out.println("Question " + QuestionNumber);
                System.out.println(Table + " X " + QuestionNumber);
                Answer = myObj.nextInt();
                if (Answer == Table * QuestionNumber)
                {
                    System.out.println(Name + " you are right, well done");
                    Right++;
                }
                else
                {
                    System.out.println(Name + " you are wrong the answer is " + Table * QuestionNumber);
                }
            }
            System.out.println("Test over " + Name + " you got " + Right + " out of " + Questions );
            System.out.print("Do you want a retest Y or N? ");
            String Choice;
            Choice = myObj.next();
            if(!Choice.equals("Y"))
            {
                Retest = false;
            }
        }
        while (Retest);
    }
}
```

Question	Answer	Marks
13	Example 15 mark answer in pseudocode. <pre> // meaningful identifier names and appropriate data structures (variables, constants and the // given arrays) to store all the data required DECLARE TotalMark : ARRAY[1:50] OF INTEGER DECLARE AverageMark : ARRAY[1:50] OF INTEGER DECLARE SubjectCounter : INTEGER DECLARE StudentCounter : INTEGER DECLARE DistinctionNo : INTEGER DECLARE MeritNo : INTEGER DECLARE PassNo : INTEGER DECLARE FailNo : INTEGER CONSTANT Distinction = 70 CONSTANT Merit = 55 CONSTANT Pass = 40 // initialisation processes for this scenario, initialising the running totals used for // grades and combined totals DistinctionNo ← 0 MeritNo ← 0 PassNo ← 0 FailNo ← 0 FOR StudentCounter ← 1 to ClassSize TotalMark[StudentCounter] ← 0 NEXT StudentCounter // programming techniques of iteration, selection, totalling, counting and output are used </pre>	

Question	Answer	Marks
13	<pre> FOR StudentCounter ← 1 to ClassSize FOR SubjectCounter ← 1 to SubjectNo TotalMark[StudentCounter] ← TotalMark[StudentCounter] + StudentMark[StudentCounter, SubjectCounter] NEXT SubjectCounter AverageMark[StudentCounter] ← INT((TotalMark[StudentCounter] / SubjectNo) + 0.5) OUTPUT "Name ", StudentName[StudentCounter] OUTPUT "Combined total mark ", TotalMark[StudentCounter] OUTPUT "Average mark ", AverageMark[StudentCounter] IF AverageMark[StudentCounter] >= Distinction THEN DistinctionNo ← DistinctionNo + 1 OUTPUT "Grade Distinction" ELSE IF AverageMark[StudentCounter] >= Merit THEN MeritNo ← MeritNo + 1 OUTPUT "Grade Merit" ELSE IF AverageMark[StudentCounter] >= Pass THEN PassNo ← PassNo + 1 OUTPUT "Grade Pass" ELSE FailNo ← FailNo + 1 OUTPUT "Grade Fail" ENDIF ENDIF ENDIF ENDIF ENDIF NEXT StudentCounter OUTPUT "Number of Distinctions ", DistinctionNo OUTPUT "Number of Merits ", MeritNo OUTPUT "Number of Passes ", PassNo OUTPUT "Number of Fails ", FailNo </pre>	

QUESTION



A programmer has been asked to design and write a program to accept inputs about each game player:

- name
- age in years
- nickname
- if the player wants to use their name or their nickname.

The program must include a validation check on the age, ensure the name and the nickname are no more than eight characters long, then output a personalised welcome message for each player.

There can be three, four or five players for each game; the input routine finishes when all the players have entered their details.

a) Give a suitable identifier name and data type for each input. Explain your choice of data type for each input.

b) Write your design for the program in pseudocode. Include the validation checks for input. Your program must include selection and iteration, use comments to show where these have been used and give the reason why they have been used.

[12]

Extend your program design to store each name and nickname in a two-dimensional array, the age in another array and the choice in another array. The index of the array will identify the player.

Use a procedure to output the personalised welcome message for each player when all players have entered their details. This output should be in ascending order of age.

a) Declare and initialise the arrays needed.

b) Write your design for the program in pseudocode.

You do not need to include the declarations for the arrays and the validation checks for input.

[12]

MARKSCHEME




```

DECLARE Name : STRING
DECLARE Age : INTEGER
DECLARE Nickname : STRING
DECLARE Choice, Another : BOOLEAN
DECLARE Player : INTEGER

Player ← 0
Another ← TRUE

REPEAT // Iteration to enter at least 3 players
    REPEAT
        OUTPUT "Enter your name, use 8 characters or less "
        INPUT Name
    UNTIL LENGTH(Name) <= 8
    REPEAT
        OUTPUT "Enter your age in years "
        INPUT Age
    UNTIL Age > 5 AND Age < 100
    REPEAT
        OUTPUT "Enter your nickname, use 8 characters or less "
        INPUT Nickname
    UNTIL LENGTH(Nickname) <= 8
    OUTPUT "Use your name (True) or Nickname (FALSE) "
    INPUT Choice
    IF Choice // Selection whether to use the name or the nickname
        THEN
            OUTPUT "Welcome ", Name, " aged ", Age
        ELSE
            OUTPUT "Welcome ", Nickname, " aged ", Age
        ENDIF
    Player ← Player + 1
    IF Player >= 3 AND Player < 6 // selection for another player
        THEN
            OUTPUT "Do you want to add another player TRUE or FALSE? "
            INPUT Another
        ENDIF
    UNTIL Player = 6 OR NOT Another

```

a)

```
DECLARE NameAndNickname : ARRAY[1:5, 1:2] OF STRING
DECLARE Age : ARRAY[1:5] OF INTEGER
DECLARE Choice : ARRAY[1:5] OF BOOLEAN
```

b)

```
// procedure definition for the welcome messages
PROCEDURE Welcome (PlayerNo : INTEGER)
    DECLARE Count, Last, TempAge: INTEGER
    DECLARE Swap, TempChoice : BOOLEAN
    DECLARE TempString : STRING
    Last ← PlayerNo

    REPEAT // bubble sort all arrays on Age in ascending order
        Swap ← FALSE
        FOR COUNT ← 1 TO Last - 1
            IF Age[Count] > Age[Count + 1]
                THEN
                    TempAge ← Age[Count]
                    Age[Count] ← Age[Count + 1]
                    Age[Count + 1] ← TempAge
                    TempChoice ← Choice [Count]
                    Choice [Count] ← Choice [Count + 1]
                    Choice [Count + 1] ← TempChoice
                    TempString ← NameAndNickname [Count,1]
                    NameAndNickname [Count,1] ← NameAndNickname [Count + 1,1]
```

```

        NameAndNickname [Count + 1,1] ← TempString
        TempString ← NameAndNickname [Count,2]
        NameAndNickname [Count,2] ← NameAndNickname [Count + 1,2]
        NameAndNickname [Count + 1,2] ← TempString
        Swap ← TRUE
    NEXT Count
    Last ← Last - 1
UNTIL (NOT Swap) or Last = 1

FOR Count ← 1 TO PlayerNo // print welcome messages
    IF Choice[Count]
        THEN
            OUTPUT "Welcome ", NameAndNickname [Count,1], " aged ", Age[Count]
        ELSE
            OUTPUT "Welcome ", NameAndNickname [Count,2], " aged ", Age[Count]
        ENDIF
    NEXT Count
ENDPROCEDURE

DECLARE Player : INTEGER
DECLARE Another : BOOLEAN
Player ← 1
Another ← TRUE
REPEAT // Iteration to enter at least 3 players
    OUTPUT "Enter your name, use 8 characters or less "
    INPUT NameAndNickname[Player,1]
    OUTPUT "Enter your age in years "
    INPUT Age[Player]
    OUTPUT "Enter your nickname, use 8 characters or less "
    INPUT NameAndNickname[Player,2]
    INPUT Choice[Player]
    Player ← Player + 1
IF Player > 3 AND Player <= 6 // selection for another player
    THEN
        OUTPUT "Do you want to add another player TRUE or FALSE? "
        INPUT Another
    ENDIF
UNTIL Player = 6 OR NOT Another
Player = Player - 1 // setting Player to the number of players entered
Welcome (Player)// procedure call

```


QUESTION



Using a programming language write a program to store the names of 50 people playing a game and their high scores. Output the names and the scores ordered from highest to lowest score and output the average score.

Your program must include and use meaningful identifier names for

- variables
- constants
- arrays
- procedures / functions.

Your program must be fully commented using your programming languages commenting feature.

[10]

MARKSCHEME



Python:

```
def FindAverage(): # function to find the average

    Total = 0

    for Counter in range(50):

        Total = Total + GamerHighScore[Counter]

    return Total / 50

def SortGamers():# function to sort names and scores in descending order

    Swap = True

    First = 0

    Last = 50

    while Swap and Last > 0:

        Swap = False

        for Counter in range(Last - 1):

            if GamerHighScore[Counter] < GamerHighScore[Counter + 1]:

                TempHighScore = GamerHighScore[Counter]

                GamerHighScore[Counter] = GamerHighScore[Counter + 1]

                GamerHighScore[Counter + 1] = TempHighScore

                TempName = GamerName[Counter]

                GamerName[Counter] = GamerName[Counter + 1]

                GamerName[Counter + 1] = TempName

                Swap = True

        Last = Last - 1

# input routine

GamerName=[]

GamerHighScore=[]

for Counter in range(50):

    GamerName.append (input("Enter next gamer's name "))

    GamerHighScore.append (int(input("Enter next gamer's high score ")))

print (GamerName, GamerHighScore)

print ("Average is ", FindAverage())

SortGamers()

print (GamerName, GamerHighScore)
```

Visual Basic:

```
Module Module1
```

```
    Dim GamerHighScore(4), Counter As Integer
```

```
    Dim GamerName(4) As String
```

```
    'input routine
```

```
    Sub Main()
```

```
        For Counter = 0 To 49
```

```
            Console.Write("Enter next gamer's name  ")
```

```
            GamerName(Counter) = Console.ReadLine()
```

```
            Console.Write("Enter next gamer's high score  ")
```

```
            GamerHighScore(Counter) = Integer.Parse(Console.ReadLine())
```

```
        Next
```

```

For Counter = 0 To 49
    Console.WriteLine(GamerName(Counter) & " " &
        GamerHighScore(Counter))
Next
Console.WriteLine("Average is " & FindAverage())
SortGamers()
For Counter = 0 To 49
    Console.WriteLine(GamerName(Counter) & " " &
        GamerHighScore(Counter))
Next
Console.ReadKey()
End Sub

' function to find average
Function FindAverage() As Decimal
    Dim Total As Integer
    Total = 0
    For Counter = 0 To 49
        Total = Total + GamerHighScore(Counter)
    Next
    Return Total / 50
End Function

' procedure to sort names and scores in descending order
Sub SortGamers()
    Dim Swap As Boolean
    Dim TempHighScore, First, Last As Integer
    Dim TempName As String
    First = 0
    Last = 49
    Do
        Swap = False
        For Counter = 0 To Last - 1
            If GamerHighScore(Counter) < GamerHighScore(Counter + 1) Then
                TempHighScore = GamerHighScore(Counter)
                GamerHighScore(Counter) = GamerHighScore(Counter + 1)
                GamerHighScore(Counter + 1) = TempHighScore
                TempName = GamerName(Counter)
                GamerName(Counter) = GamerName(Counter + 1)
                GamerName(Counter + 1) = TempName
                Swap = True
            End If
        Next
        Last = Last - 1
    Loop Until Not Swap Or Last = 1
End Sub

```


Java:

```
import java.util.Scanner;

class Ch8Q6Java {

    static int GamerHighScore [] = new int[5];
    static String GamerName [] = new String[5];

    static double FindAverage(){
        int Counter, Total;
        Total = 0;
        for ( Counter = 0; Counter <= 4; Counter ++ ) {
            Total = Total + GamerHighScore[Counter];
        }
        return Total / 5;
    }

    static void SortGamers(){
        int TempHighScore, First, Last, Counter;
        First = 0;
        Last = 4;
        boolean Swap;
        String TempName;
        do{
            Swap = false;
            for (Counter = 0; Counter<= Last - 1; Counter ++){
                if (GamerHighScore[Counter] < GamerHighScore[Counter + 1])
                {
                    TempHighScore = GamerHighScore[Counter];
                    GamerHighScore[Counter] = GamerHighScore[Counter + 1];
                    GamerHighScore[Counter + 1] = TempHighScore;
                    TempName = GamerName[Counter];
                    GamerName[Counter] = GamerName[Counter + 1];
                    GamerName[Counter + 1] = TempName;
                    Swap = true;
                }
            }
            Last = Last - 1;
        } while (!Swap || Last > 1);
    }

    public static void main(String args[]){
        Scanner myObj = new Scanner(System.in);
        int Counter;
        for (Counter = 0; Counter <= 4; Counter ++ ) {
```



```
System.out.print("Enter next gamer's name ");
GamerName[Counter] = myObj.next();
System.out.print("Enter next gamer's high score ");
GamerHighScore[Counter] = myObj.nextInt();
}
for (Counter = 0; Counter <= 4; Counter++) {
System.out.println(GamerName[Counter] + " " + GamerHighScore[Counter]);
}
System.out.println("Average is " + FindAverage());
SortGamers();
for (Counter = 0; Counter <= 4; Counter++) {
System.out.println(GamerName[Counter] + " " + GamerHighScore[Counter]);
}
}
}
```

QUESTION



The one-dimensional array `BabyName[]` contains the names of the babies in a nursery.

Another one-dimensional array `ParentPhone []` contains the phone numbers for the parents of the baby.

A final one-dimensional array `BabyAge[]` contains the baby's age in months.

The position of each baby's data in the three arrays is the same; for example, the baby in position 3 in `BabyName`, `ParentPhone` and `BabyAge[]` is the same.

Write and test a program that meets the following requirements:

1. uses procedures to display these lists:
 - parent phone numbers and baby names in alphabetic order of baby name
 - baby names for babies aged under three months
2. uses a procedure to display all the details for a baby, with the name used as a parameter
3. uses a procedure to update the details for a baby, with the name used as a parameter.

You must use pseudocode or program code with local and global variables and add comments to explain how your code works.

All inputs and outputs must contain suitable messages.

MARKSCHEME




```

def YoungBabies(): # procedure to find babies under 3 months
    print("Names of babies under 3 months")
    for Counter in range(Length):
        if BabyAge[Counter] < 3:
            print (BabyName[Counter])

def FindBaby(Baby): # procedure to find a baby
    print("Baby's details")
    for Counter in range(Length):
        if BabyName[Counter] == Baby:
            print (BabyName[Counter], BabyAge[Counter], ParentPhone[Counter])

```

```

def UpdateBaby(Baby): # procedure to update a baby's details
    print("Baby's details")
    for Counter in range(Length):
        if BabyName[Counter] == Baby:
            BabyAge[Counter] = int(input("Enter baby's age in months "))
            ParentPhone[Counter] = input("Enter parents' phone number ")

def SortBabys(): # procedure to sort babies in ascending order of age
    Swap = True
    First = 0
    Last = 5
    while Swap and Last > 0:
        Swap = False
        for Counter in range(Last - 1):
            if BabyName[Counter] > BabyName[Counter + 1]:
                TempBabyName = BabyName[Counter]
                BabyName[Counter] = BabyName[Counter + 1]
                BabyName[Counter + 1] = TempBabyName
                TempAge = BabyAge[Counter]
                BabyAge[Counter] = BabyAge[Counter + 1]
                BabyAge[Counter + 1] = TempAge
                TempPhone = ParentPhone[Counter]
                ParentPhone[Counter] = ParentPhone[Counter + 1]
                ParentPhone[Counter + 1] = TempPhone
                Swap = True
        Last = Last - 1

YoungBabies()

FindBaby(input("Enter baby's name "))

UpdateBaby(input("Enter baby's name "))

SortBabys()

print (BabyName, BabyAge, ParentPhone)

```


QUESTION



2D array called TeamPoints that stores the points for every match for every team. 1D array called TeamName that stores the names of every team at the same index location as in TeamPoints.

A constant LeagueSize with the number of teams. A constant MatchNo with the number of matches played by each team.

Points in the 2D array TeamPoints are awarded based on the following criteria:

- An away win is 3 points
- A home win is 2 points
- A draw is 1 point
- A loss is 0 points

You must:

- Calculate and store the total points scored
- Calculate and display the total points for all matches in each category (away win, home win, draw, loss)
- Calculate and display the total team points, total team points for each category and name for each of the teams
- Find and display the name of the team with the highest and lowest points

MARKSCHEME



(Unofficial Python Markscheme by S_Skamr ~ Past Paper hasn't been released yet)

Note: This code makes use of helper functions and dictionaries
(Discouraged for IGCSE Syllabus)

```
"""
```

```
MAPPINGS
```

```
0: Loss
1: Draw
2: Home Win
3: Away Win
```

They start with 0 so it can be easily used to index into a list that stores every count for the mappings - helpers.

```
"""
```

```
# Test data generation starts (not required) -----
```

```
from random import randint
```

```
# random test team names
```

```
teams = ["Berkeley", "Princeton", "Stanford", "Michigan", "CalTech",  
"Duke", "Yale", "Cambridge", "Oxford", "Imperial", "Kings", "Harvard",  
"NYU", "UC Davis"]
```

```
# number of mappings - this should never change as the number of  
categories are a total of 4 always
```

```
MAP_COUNT = 4
```

```
# number of teams
```

```
TEAM_COUNT = len(teams)
```

```
# number of matches
```

```
MATCH_COUNT = randint(4,12)
```

```
# Generate a list of random numbers from 0-3 for the number of matches  
for each team - 1 list of random numbers for every team
```

```
points = [[randint(0, 3) for i in range(MATCH_COUNT)] for x in  
range(TEAM_COUNT)]
```

```
# Test data generation over -----
```



```

# Generate an empty helper for all category mappings to maintain their
counts
def genHelper():
    return [0] * MAP_COUNT

# each point value maps to a certain category of outcome - this function
increments the helper value for that outcome; each helper is essentially a
list where the index corresponds to the point value
def incrementHelper(point, helper):
    # increment the helper's value at index 'point'
    helper[point] += 1
    # return the full helper back
    return helper

# Maps each index to the required message when printing - for eg. an index
of 1 corresponds to the returned message "Draws" which can be used to
display the required details for draws
def mapMsg(index):
    return ["Losses", "Draws", "Home wins", "Away wins"][index]

# Set total points to 0 and generate a helper for totalCategory
(totalCategory counts the total count of each category)
totalPoints = 0
totalCategory = genHelper()

# Setting highest and lowest to [Name, Point] format with corresponding
values for 2nd index
highest = [None, 2**-64]
lowest = [None, 2**64]

# Enumerate self implementation - uses a generator to iterate over an
iterable while maintaining the count for index 'i'
def enumerateOver(iterObj, s=0):
    i = s
    for element in iterObj:
        yield i, element
        i += 1

```

```

# Loop over the index and matches returned by enumerating over the points
list
for i, teamMatches in enumerateOver(points):
    # Generate a helper for the current team's category counts
    teamHelper = genHelper()
    # Current teams total points
    teamTotalPoints = 0
    # Points for every match of every team
    for points in teamMatches:
        # Add the points to the team's total points and the total Global
points
        teamTotalPoints += points
        totalPoints += points
        # Increment the total category based helper and the current teams
helper as required
        totalCategory, teamHelper = incrementHelper(points,
totalCategory), incrementHelper(points, teamHelper)
        # Print the team's name as corresponding to the current index
        print(f"Team name: {teams[i]}")
        # Use the number of mappings and the 'mapMsg' function to display
Losses, Draws, Home wins, and Away wins for every team
        for j in range(MAP_COUNT):
            print(f"Team {mapMsg(j)}: {teamHelper[j]}")
        # Display the team's total points
        print(f"Team total points: {teamTotalPoints}")

        # Check if the current teams points belong to highest or lowest and
set highest or lowest to the current teams name and points if so
        if teamTotalPoints > highest[1]:
            highest = [teams[i], teamTotalPoints]
        if teamTotalPoints < lowest[1]:
            lowest = [teams[i], teamTotalPoints]

        # Print a divider for aesthetics
        print("---")

# For each of the mappings print out the totals
for j in range(MAP_COUNT):
    print(f"Total {mapMsg(j)}: {totalCategory[j]}")
print("---")

# Print the team with the lowest and highest points and their names
print(f"Lowest scoring team with {lowest[1]} points is {lowest[0]}.")
print(f"Highest scoring team with {highest[1]} points is {highest[0]}.")

```